# EVERGAME 360 INTELLIGENCE DASHBOARD

## Implementation Guide & Documentation

**Version:** 1.0

**Date:** November 19, 2025

**Component:** EVERGAME360_Intelligence_Dashboard.jsx

**Framework:** React 18.2.0 + Tailwind CSS

---

## 🎯 OVERVIEW

The EVERGAME 360 Intelligence Dashboard is a comprehensive, real-time operational intelligence interface for NFL game day operations. It integrates temporal orchestration (M1-M6 phases), venue operations tracking, GDA certification monitoring, system health matrices, and NIN intelligence insights into a single, unified executive command center.

## Key Features

✅ **Real-Time Temporal Orchestration** - M1-M6 phase tracking with visual progress

✅ **League-Wide Venue Monitoring** - 32 NFL stadiums with live status

✅ **GDA Certification Tracking** - 238+ GDAs with 4-tier certification system

✅ **System Health Matrix** - 9-11 critical systems monitored per venue

✅ **NIN Intelligence Insights** - AI-powered predictions and optimizations

✅ **Executive KPIs** - ROI, compliance, financial exposure, risk assessment

✅ **Multi-View Modes** - Executive, Operational, Intelligence perspectives

✅ **Responsive Design** - Works on desktop, tablet, and mobile

✅ **Real-Time Updates** - Auto-refresh with WebSocket support

---

# 📋 QUICK START

## Option 1: Use in Lovable (Recommended)

1. **Create New Component in Lovable:**

```
# In your Lovable project
- Create new file: components/EVERGAME360Dashboard.jsx
- Copy the entire dashboard code into this file
```

2. **Import in Your App:**

```
import EVERGAME360Dashboard from './components/
EVERGAME360Dashboard';

function App() {
  return <EVERGAME360Dashboard />;
}
```

3. **Ensure Dependencies:**
4. React 18.2.0+
5. lucide-react (icons)
6. Tailwind CSS configured
7. **Run:**

```
npm run dev
```

# Option 2: Standalone React Project

1. **Create React App:**

```
npx create-react-app evergame-dashboard
cd evergame-dashboard
```

2. **Install Dependencies:**

```
npm install lucide-react
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p
```

3. **Configure Tailwind:**

```
// tailwind.config.js
module.exports = {
  content: [
    "./src/**/*.{js,jsx,ts,tsx}",
  ],
  theme: {
    extend: {},
  },
  plugins: [],
}
```

4. **Add Tailwind to CSS:**

```
/* src/index.css */
@tailwind base;
@tailwind components;
@tailwind utilities;
```

5. **Copy Dashboard Component:**
6. Copy EVERGAME360_Intelligence_Dashboard.jsx to src/
7. Import in App.js
8. **Run:**

```
npm start
```

# Option 3: Integration with Simulation Environment

```
# If using Docker Compose simulation from earlier
```

```
# Add dashboard to docker-compose.simulation.yml

dashboard-ui:
  image: node:18-alpine
  working_dir: /app
  volumes:
    - ./src/ui:/app/src
    - ./package.json:/app/package.json
  command: npm run dev
  ports:
    - "3000:3000"
  environment:
    - VITE_API_URL=http://evergame-core:8080

---
```

# 🏗️ ARCHITECTURE

## Component Structure

```
EVERGAME360Dashboard (Main Container)
│
├── Header Section
│   ├── Title & Branding
│   ├── Real-Time Clock
│   └── View Mode Selector
│
├── KPI Bar (League-Wide Metrics)
│   ├── Active Venues
│   ├── GDA Certification Rate
│   ├── Intelligence Score
│   ├── Systems Operational
│   ├── Financial Exposure
```

```
│   ├── ROI Multiple
│   └── Critical Issues
│
├── Temporal Orchestration Timeline
│   ├── M1: DEFINE (T-6h to T-5h)
│   ├── M2: INFRASTRUCTURE (T-5h to T-3h)
│   ├── M3: NETWORK (T-3h to T-2h)
│   ├── M4: OPERATIONS (T-2h to T-1h)
│   ├── M5: GAME TIME (T-1h to Final)
│   └── M6: DEBRIEF (Final to +3h)
│
├── View Modes (Conditional Rendering)
│   │
│   ├── Executive View
│   │   ├── Venue Grid (6 active venues shown)
│   │   └── System Health Matrix (9 systems)
│   │
│   ├── Operational View
│   │   ├── GDA Position Detail
│   │   ├── System Status Detail
│   │   ├── Certification Breakdown
│   │   └── Temporal Progress Detail
│   │
│   └── Intelligence View
│       ├── NIN Insights (Predictions, Optimizations, Compliance)
│       ├── Certification Analysis
│       └── ROI Metrics
│
└── Footer
    └── System Info & Last Update
```

# Data Flow

```
User Interaction
      ↓
State Management (React Hooks)
```

```
      ↓
Component Re-render
      ↓
UI Update

[In Production:]
API Endpoint → WebSocket → State Update → UI Refresh
```

---

# 🎨 DESIGN SYSTEM

## Color Palette

```
/* Status Colors */
--status-optimal: #10b981 (green)
--status-ready: #3b82f6 (blue)
--status-at-risk: #f59e0b (yellow)
--status-critical: #ef4444 (red)

/* Accent Colors */
--accent-cyan: #06b6d4
--accent-blue: #3b82f6
--accent-green: #10b981

/* Background */
--bg-primary: gradient(gray-900, blue-900)
--bg-card: rgba(255, 255, 255, 0.05)

/* Text */
--text-primary: #ffffff
--text-secondary: #9ca3af
```

## Typography

```
/* Headers */
h1: 36px, bold
h2: 24px, bold
h3: 18px, bold

/* Body */
p: 14px, regular
small: 12px, regular

/* Monospace (for times/metrics) */
time: 24px, mono, bold
metric: 20px, mono, bold
```

## Spacing System

- **Gap between cards:** 16px (1rem)
- **Card padding:** 24px (1.5rem)
- **Section margin:** 32px (2rem)

---

# 🔌 API INTEGRATION

## Expected API Endpoints

```
// In production, replace mock data with API calls

// League-wide metrics
GET /api/v1/league/metrics
Response: {
  totalVenues: 32,
  activeGames: 14,
  totalGDAs: 238,
  certifiedGDAs: 235,
  certificationRate: 98.7,
  // ... other metrics
}

// Venue data
GET /api/v1/venues
Response: [
  {
    id: 'metlife',
    name: 'MetLife Stadium',
    intelligenceScore: 94.2,
    // ... other venue data
  },
  // ...
]

// Temporal phases
GET /api/v1/temporal/phases
Response: [
  {
    id: 'M1',
    name: 'DEFINE',
```

```
    status: 'complete',
    // ...
  },
  // ...
]

// System health
GET /api/v1/systems/health
Response: [
  {
    name: 'IVRS',
    status: 'operational',
    venues: 32,
    issues: 0
  },
  // ...
]

// NIN insights
GET /api/v1/nin/insights
Response: [
  {
    type: 'PREDICTION',
    severity: 'warning',
    message: '...',
    // ...
  },
  // ...
]
```

## WebSocket Integration (Real-Time Updates)

```
// Add to dashboard component for live updates

useEffect(() => {
  const ws = new WebSocket('ws://localhost:8080/ws');
```

```
  ws.onmessage = (event) => {
    const update = JSON.parse(event.data);

    switch(update.type) {
      case 'VENUE_UPDATE':
        updateVenueData(update.data);
        break;
      case 'PHASE_TRANSITION':
        updatePhaseData(update.data);
        break;
      case 'SYSTEM_STATUS':
        updateSystemHealth(update.data);
        break;
      case 'NIN_INSIGHT':
        addNINInsight(update.data);
        break;
    }
  };

  return () => ws.close();
}, []);
```

---

# 🎯 CUSTOMIZATION GUIDE

## Adding New Venue

```
// In venues array, add:
{
  id: 'new-venue',
  name: 'New Stadium',
```

```
  location: 'City, State',
  homeTeam: 'Team Name',
  visitorTeam: 'Opponent',
  gameTime: '13:00',
  timeToKickoff: 'T-2h 30m',
  currentPhase: 'M3',
  intelligenceScore: 88.5,
  certificationRate: 95.0,
  systemsOnline: 10,
  systemsTotal: 11,
  gdasAssigned: 16,
  gdasCheckedIn: 14,
  criticalIssues: 0,
  warnings: 2,
  financialExposure: 125000,
  status: 'READY',
  statusColor: 'bg-blue-500/20 text-blue-400 border-blue-500/30'
}
```

## Adding New System to Health Matrix

```
// In systemHealth array, add:
{
  name: 'NEW_SYSTEM',
  status: 'operational', // or 'degraded', 'failed'
  venues: 32,
  issues: 0
}
```

## Adding New NIN Insight

```
// In ninInsights array, add:
{
```

```
  type: 'PREDICTION', // or 'OPTIMIZATION', 'COMPLIANCE'
  severity: 'warning', // or 'info', 'success'
  venue: 'Venue Name',
  system: 'System Name',
  message: 'Insight message here',
  recommendation: 'Action to take',
  confidence: 85.3,
  financialImpact: 250000 // positive = exposure, negative = savings
}
```

## Changing Color Scheme

```
// Replace Tailwind classes in components

// Current: Blue/Cyan theme
className="text-blue-400"
className="bg-blue-500"

// Alternative: Green theme
className="text-green-400"
className="bg-green-500"

// Alternative: Purple theme
className="text-purple-400"
className="bg-purple-500"
```

---

# 📊 DATA MODELS

## Venue Model

```typescript
interface Venue {
  id: string;
  name: string;
  location: string;
  homeTeam: string;
  visitorTeam: string;
  gameTime: string; // HH:MM format
  timeToKickoff: string; // "T-Xh Ym"
  currentPhase: 'M1' | 'M2' | 'M3' | 'M4' | 'M5' | 'M6';
  intelligenceScore: number; // 0-100
  certificationRate: number; // 0-100
  systemsOnline: number;
  systemsTotal: number;
  gdasAssigned: number;
  gdasCheckedIn: number;
  criticalIssues: number;
  warnings: number;
  financialExposure: number; // USD
  status: 'OPTIMAL' | 'READY' | 'AT RISK' | 'CRITICAL';
  statusColor: string; // Tailwind classes
}
```

## Temporal Phase Model

```typescript
interface TemporalPhase {
  id: 'M1' | 'M2' | 'M3' | 'M4' | 'M5' | 'M6';
  name: string;
  time: string; // "T-Xh to T-Yh"
```

```
  icon: LucideIcon;
  status: 'complete' | 'active' | 'pending';
  venuesComplete: number;
  venuesTotal: number;
  color: string; // Tailwind text color
}
```

## NIN Insight Model

```
interface NINInsight {
  type: 'PREDICTION' | 'OPTIMIZATION' | 'COMPLIANCE';
  severity: 'warning' | 'info' | 'success';
  venue: string;
  system: string;
  message: string;
  recommendation: string;
  confidence: number; // 0-100
  financialImpact: number; // USD (positive = exposure, negative =
savings)
}
```

---

# 🚀 DEPLOYMENT

## Production Build

```
# Build for production
npm run build
```

```
# Serve static files
npx serve -s build -p 3000

# Or deploy to hosting platform
# Vercel
vercel --prod

# Netlify
netlify deploy --prod

# AWS S3 + CloudFront
aws s3 sync build/ s3://your-bucket-name
```

## Environment Variables

```
# .env.production
REACT_APP_API_URL=https://api.evergame360.com
REACT_APP_WS_URL=wss://ws.evergame360.com
REACT_APP_ENV=production
```

## Docker Deployment

```
# Dockerfile
FROM node:18-alpine as build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/build /usr/share/nginx/html
EXPOSE 80
```

```
CMD ["nginx", "-g", "daemon off;"]
```

---

# 🧪 TESTING

## Unit Tests

```
import { render, screen } from '@testing-library/react';
import EVERGAME360Dashboard from './EVERGAME360Dashboard';

test('renders dashboard title', () => {
  render(<EVERGAME360Dashboard />);
  const titleElement = screen.getByText(/EVERGAME 360/i);
  expect(titleElement).toBeInTheDocument();
});

test('displays correct number of venues', () => {
  render(<EVERGAME360Dashboard />);
  const venueCards = screen.getAllByTestId('venue-card');
  expect(venueCards).toHaveLength(6);
});
```

## Integration with Simulation API

```
// Test with Docker Compose simulation environment
const API_BASE = 'http://localhost:8080/api/v1';

async function fetchRealData() {
```

```
  const metrics = await fetch(`${API_BASE}/league/metrics`).then(r
=> r.json());
  const venues = await fetch(`${API_BASE}/venues`).then(r =>
r.json());
  const phases = await fetch(`${API_BASE}/temporal/phases`).then(r
=> r.json());

  return { metrics, venues, phases };
}
```

---

# 🎓 USAGE SCENARIOS

## Scenario 1: Executive Monitoring (Pre-Game)

**User:** NFL CTO

**View:** Executive

**Time:** T-2h before games

**Actions:**

1. Opens dashboard
2. Reviews league-wide KPIs
3. Checks temporal progress (M4 active)
4. Identifies Arrowhead Stadium at CRITICAL status
5. Clicks venue for details
6. Switches to Intelligence view to see NIN prediction

7. Contacts on-site NFL Lead to deploy Tier 4 GDA

## Scenario 2: Operational Coordination (Game Time)

**User:** NFL IT Lead

**View:** Operational

**Time:** T-30m before kickoff

**Actions:**

1. Opens dashboard
2. Selects venue (MetLife Stadium)
3. Views GDA positions (all 16 checked in ✓)
4. Confirms all systems operational
5. Checks Tier 2/3 certification rates
6. Monitors temporal progress (M4 → M5 transition)

## Scenario 3: Intelligence Analysis (Post-Game)

**User:** Operations Analyst

**View:** Intelligence

**Time:** Post-game debrief

**Actions:**

1. Opens dashboard
2. Switches to Intelligence view
3. Reviews NIN insights from game day

4. Analyzes certification compliance (98.9%)
5. Calculates ROI metrics (2,285x multiple)
6. Generates executive report

---

# 🔧 TROUBLESHOOTING

## Issue: Dashboard Not Loading

**Solution:**

```
# Check dependencies
npm install

# Clear cache
rm -rf node_modules package-lock.json
npm install

# Verify Tailwind config
npx tailwindcss -i ./src/index.css -o ./dist/output.css --watch
```

## Issue: Icons Not Displaying

**Solution:**

```
# Reinstall lucide-react
npm uninstall lucide-react
```

```
npm install lucide-react@latest
```

## Issue: Real-Time Updates Not Working

**Solution:**

```
// Check WebSocket connection
const ws = new WebSocket('ws://localhost:8080/ws');
ws.onopen = () => console.log('Connected');
ws.onerror = (error) => console.error('WebSocket error:', error);
```

## Issue: Tailwind Classes Not Applied

**Solution:**

```
// Verify tailwind.config.js includes JSX files
module.exports = {
  content: [
    "./src/**/*.{js,jsx,ts,tsx}",
  ],
  // ...
}
```

---

# 📚 RESOURCES

- **React Documentation:** https://react.dev
- **Tailwind CSS:** https://tailwindcss.com

- **Lucide Icons:** https://lucide.dev
- **Lovable Platform:** https://lovable.dev

---

# 🎉 NEXT STEPS

1. **Integrate with Live Data**
2. Connect to EVERGAME 360 API endpoints
3. Implement WebSocket for real-time updates
4. Add authentication/authorization
5. **Enhance Functionality**
6. Add filtering and search
7. Implement drill-down details
8. Add export capabilities (PDF, Excel)
9. **Mobile Optimization**
10. Test on mobile devices
11. Optimize touch interactions
12. Add responsive breakpoints
13. **Performance**
14. Implement data caching
15. Add loading states
16. Optimize re-renders

---

**Document Version:** 1.0

**Last Updated:** November 19, 2025

**Component Version:** v1.0

**Maintainer:** EVERGAME 360 Development Team